Document number: P0704R1
Date: 2017-07-10
Audience: Evolution Working Group
Reply-To: Barry Revzin <barry.revzin@gmail.com>
(minor edits from CWG by Davis Herring <herring@lanl.gov>)

Fixing const-qualified pointers to members

Motivation

Consider the following example:

```
struct X { void foo() const&; };

X{}.foo();        // this is okay
(X{}.*&X::foo)(); // this is ill-formed
```

The second expression is ill-formed due to the wording in [expr.mptr.oper]/6:

> In a `.*` expression whose object expression is an rvalue, the program is ill-formed if the second operand is a pointer to member function with *ref-qualifier* `&`.

but the first expression is fine because we can bind an rvalue `X` to the implicit object parameter which is of type `X const&`. That seems inconsistent, since the two expressions fundamentally mean the same thing.

Proposal

Modify the wording of [expr.mptr.oper]/6 to allow pointer-to-member expressions when the object is an rvalue and the pointer to member is `const&`-qualified, as follows:

> In a `.*` expression whose object expression is an rvalue, the program is ill-formed if the second operand is a pointer to member function ~~with~~ whose *ref-qualifier* is `&`, unless its *cv-qualifier-seq* is `const`.

Acknowledgements

Thanks to Mike Spertus for suggesting that this should be a proposal.